

Disciplina: Pesquisa e Ordenação de Dados
Curso: Sistemas de Informação
Aluno: Ricardo Castelli Rupp

1. Introdução

A ordenação de dados é uma operação importante na área da computação, pois permite organizar informações de forma eficiente para facilitar buscas, consultas e análises. Em aplicações reais, entretanto, o volume de dados pode ultrapassar a capacidade da memória RAM disponível, tornando inviável carregar todo o conjunto de dados simultaneamente.

Para lidar com esse problema, são utilizadas técnicas de ordenação externa, que permitem processar arquivos diretamente em dispositivos de armazenamento secundário, como HDs e SSDs, utilizando apenas parte da memória principal durante a execução.

Este trabalho tem como objetivo apresentar uma pesquisa sobre técnicas de ordenação externa, analisar aplicações práticas dessa abordagem e desenvolver uma implementação funcional em Python utilizando uma base de dados simulada.

2. Pesquisa sobre Ordenação Externa

A ordenação externa reúne métodos desenvolvidos para trabalhar com conjuntos de dados que excedem a capacidade da memória principal.

Durante a pesquisa foram analisadas as seguintes técnicas:

External Merge Sort

Consiste em dividir o arquivo em blocos menores que possam ser carregados em memória. Cada bloco é ordenado individualmente e armazenado temporariamente em disco. Ao final, os blocos são intercalados para gerar o arquivo ordenado.

Replacement Selection

Técnica que utiliza estruturas de prioridade para gerar blocos iniciais maiores, reduzindo a quantidade de arquivos temporários criados.

Multiway Merge

Realiza a intercalação simultânea de vários arquivos temporários, diminuindo o número de leituras e gravações necessárias.

Polyphase Merge Sort

Método que busca otimizar o uso do armazenamento secundário por meio de uma distribuição mais eficiente dos arquivos temporários.

3. Aplicações Práticas

A ordenação externa possui diversas aplicações em ambientes que trabalham com grande volume de informações.

Bancos de dados

Utilizada em processos de indexação, consultas e organização de tabelas extensas.

Sistemas distribuídos

Empregada no processamento e consolidação de dados provenientes de diferentes servidores.

Big Data

Aplicada em sistemas responsáveis pelo processamento contínuo de grandes quantidades de dados.

Mecanismos de busca

Utilizada na criação e atualização de índices que permitem localizar informações rapidamente.

4. Técnica Escolhida

Para a implementação prática foi escolhida a técnica **External Merge Sort**.

A escolha ocorreu por apresentar uma estrutura relativamente simples de implementar e por demonstrar claramente os conceitos envolvidos na ordenação externa.

Seu funcionamento ocorre em duas etapas principais:

- **Divisão:** os dados são separados em blocos menores e ordenados individualmente;
- **Intercalação:** os blocos ordenados são combinados para produzir o arquivo final.

5. Metodologia e Ambiente de Testes

Para validar o funcionamento do algoritmo foi criada uma base simulada em Python com as seguintes características:

- Quantidade de registros: 500.000
- Tipo de dados: números inteiros
- Intervalo de valores: 1 até 1.000.000

Essa base permitiu observar o comportamento do algoritmo em um cenário com volume significativo de dados.

Implementação

O código foi desenvolvido utilizando apenas bibliotecas da biblioteca padrão do Python.

Bibliotecas utilizadas:

os

Responsável pelo gerenciamento das pastas e arquivos temporários.

heapq

Utilizada para implementar a fila de prioridade durante a intercalação dos blocos.

time

Empregada para medir o tempo de execução.

6. Resultados Obtidos

Foram realizados testes utilizando diferentes tamanhos de entrada.

Cenário	Registros	Tempo
Teste inicial	10.000	0,03s
Teste principal	500.000	1,39s

Os testes mostraram crescimento proporcional do tempo de execução conforme o aumento da quantidade de registros.

Comparação com o método sort()

Para fins de comparação, foi utilizado o método nativo sort() do Python.

- External Merge Sort: **1,39 s**
- sort() do Python: **0,26 s**

Apesar do desempenho superior do método interno para esse volume de dados, ele depende do carregamento completo da base em memória. Já a ordenação externa mantém funcionamento mesmo quando o conjunto de dados excede a RAM disponível.

7. Desafios, Vantagens e Limitações

Desafios observados

- limitação da memória principal;
- custo das operações de leitura e escrita em disco;
- gerenciamento dos arquivos temporários;
- definição adequada do tamanho dos blocos.

Vantagens

- suporte a grandes volumes de dados;
- menor dependência da memória RAM;

- possibilidade de escalabilidade.

Limitações

- desempenho inferior para arquivos menores;
- dependência da velocidade do disco;
- necessidade de armazenamento temporário.

8. Cenários de Uso

A técnica estudada pode ser aplicada em:

- processamento de logs;
- análise financeira;
- sistemas hospitalares;
- comércio eletrônico;
- monitoramento por sensores;
- processamento de dados climáticos.

9. Prints do funcionamento

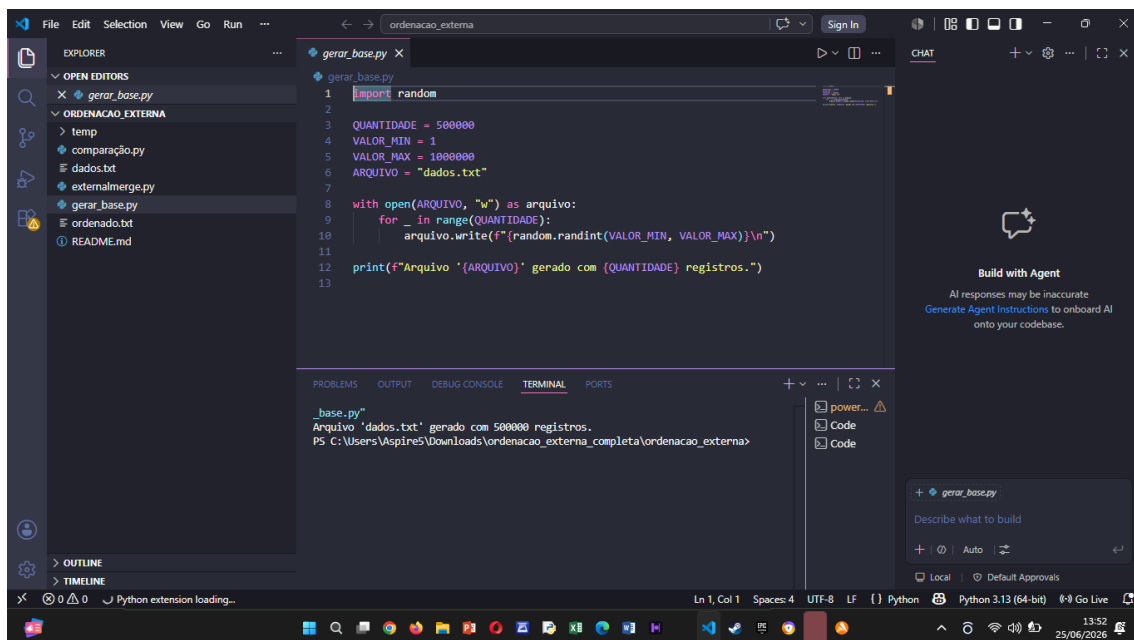


Figura 1 – Geração da base simulada utilizada nos testes

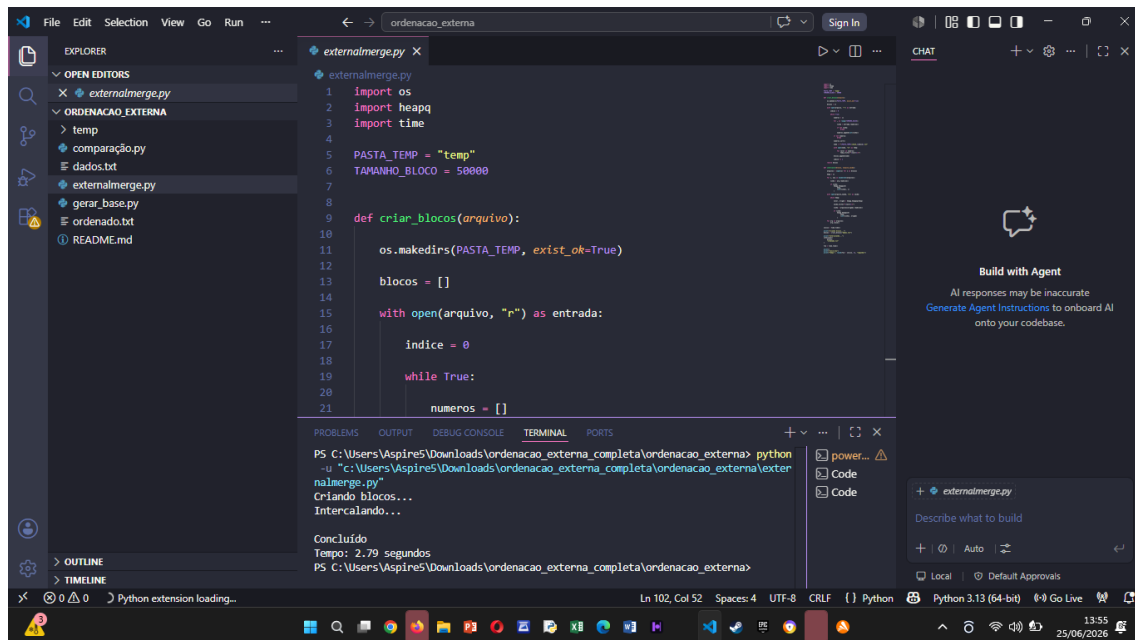


Figura 2 – Execução do algoritmo External Merge Sort

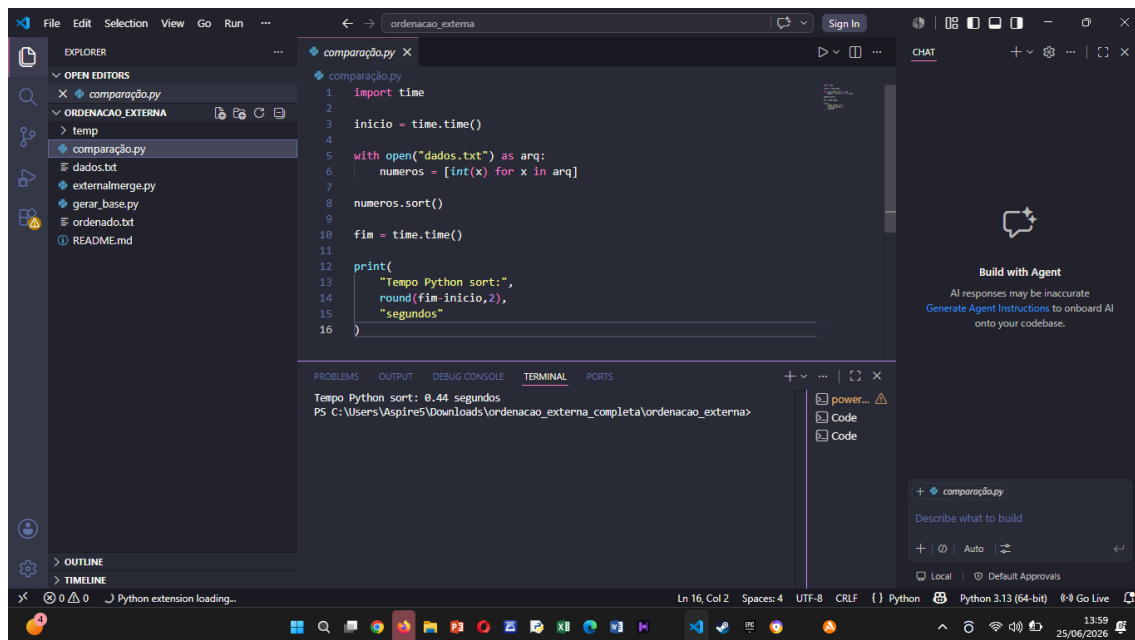


Figura 3 – Comparação entre ordenação externa e ordenação em memória

10. Conclusão

A realização deste trabalho permitiu compreender os princípios da ordenação externa e observar sua aplicação prática por meio de uma implementação em Python.

Os resultados obtidos demonstraram que, embora exista um custo adicional relacionado ao acesso em disco, essa abordagem continua sendo adequada para cenários em que o volume de dados ultrapassa os limites da memória principal.

10. Estudo de Caso – Base de 100 GB

Considerando uma base de 100 GB em um computador com apenas 4 GB de RAM, seria necessário reduzir o tamanho dos blocos processados e aumentar o número de etapas de intercalação.

Nesse cenário, os dados seriam divididos em arquivos temporários menores, processados gradualmente e posteriormente combinados até formar o arquivo final, permitindo concluir a ordenação sem exceder o limite de memória disponível.

Referências

CORMEN, Thomas H. et al. *Algoritmos: Teoria e Prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.

GeeksforGeeks. *External Sorting*.

Python Software Foundation. *Python Documentation*.